

Quiz 4

● Graded

Student

Luis Méndez Lázaro

Total Points

16.5 / 28.5 pts

Question 1

DeepLab v3+

1.5 / 1.5 pts

✓ + 1.5 pts Correcto!

+ 0 pts Respuesta: b) DeepLabV3+ combina el uso de ASPP para extraer características multiescala con un módulo de decoder que integra explícitamente low-level features para refinar las segmentaciones, superando las limitaciones de PSPNet en la precisión de los bordes.

La ventaja principal de DeepLabV3+ sobre PSPNet en la fusión de características es que DeepLabV3+ combina un módulo ASPP para capturar información multiescala con un decoder que integra explícitamente low-level features, lo que permite refinar mejor los bordes y los detalles de la segmentación.

Question 2

InfoNCE

0 / 1.5 pts

+ 1.5 pts Correcto!

✓ **+ 0 pts Respuesta: e)** InfoNCE maximiza la probabilidad de identificar correctamente el par positivo entre múltiples distractores, operando en un espacio probabilístico que refleja la información mutua, a diferencia de Triplet Loss que se centra en empujar a las muestras negativas a distancias mayores sin considerar la distribución global.

La diferencia principal entre InfoNCE y Triplet Loss radica en que InfoNCE formula la tarea de aprendizaje como un problema de clasificación (usando una normalización tipo softmax) donde se intenta identificar la pareja positiva correcta entre múltiples distractores negativos, mientras que Triplet Loss se basa en una función de margen que simplemente empuja al negativo a una distancia mayor que la del positivo con respecto al ancha (imagen original).

Question 3

Softmax

1.5 / 1.5 pts

✓ + 1.5 pts Correcto!

+ 0 pts **Respuesta: c)** Aumentar τ suaviza la distribución, reduciendo las diferencias entre las probabilidades asignadas a cada clase y haciendo que la distribución sea más uniforme.

El softmax se define como:

$$\text{Softmax}_i(z) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

Si τ aumenta, entonces $\frac{z_i}{\tau}$ se reduce, es decir, las diferencias entre los z_i se suavizan (se vuelven relativamente más pequeñas tras la división). Esto provoca que los valores $\exp(z_i/\tau)$ se acerquen más entre sí, reduciendo la diferencia entre la clase más probable y el resto. Entonces, la distribución resultante es más uniforme.

Por otro lado, si τ disminuye (por ejemplo, $\tau < 1$), se exageran las diferencias entre los logits (pues se divide por un número pequeño), haciendo que el máximo logit resalte, y la distribución se vuelva más concentrada.

Question 4

Contrastive learning

3 / 5 pts

+ 5 pts Correcto!

+ 0 pts La función loss en contrastive learning es fundamental para definir cómo se aprenden las representaciones, ya que busca acercar en el espacio de características a los pares positivos y alejar a los negativos. La definición de la distancia influye directamente en cómo se perciben las similitudes y diferencias entre muestras. Una métrica adecuada debe ser coherente con la estructura inherente de los datos y con la tarea a resolver, de modo que se pueda capturar de manera efectiva la semántica de los datos.

El parámetro temperature modula la distribución de similitudes, controlando la suavidad con la que se penalizan las discrepancias entre pares. Valores bajos de temperature agudizan el contraste entre los pares positivos y negativos, forzando al modelo a generar representaciones más discriminativas; mientras que valores más altos suavizan estas diferencias, permitiendo cierta flexibilidad. Por lo tanto, el ajuste de este parámetro es crucial para equilibrar la capacidad del modelo de distinguir diferencias relevantes sin sobreajustar el ruido presente en los datos.

✓ + 3 pts Click here to replace this description.

+ 1.5 pts Click here to replace this description.

+ 4 pts Click here to replace this description.

+ 5 pts Correcto!

+ 0 pts 1. Gradientes del mapa de profundidad $|\partial_x d_t^*|$ y $|\partial_y d_t^*|$

- Representan las variaciones en la profundidad predicha a lo largo de los ejes x e y .
- Si estos valores son grandes, significa que hay cambios bruscos en la profundidad, lo que podría indicar bordes o ruido en la predicción.
- Si estos valores son pequeños, significa que la profundidad es suave y continua, lo que es deseable en regiones uniformes.

2. Gradientes de la imagen $|\partial_x I_t|$ y $|\partial_y I_t|$

- Representan los cambios de intensidad en la imagen original en las direcciones x e y .
- Un gradiente grande indica la presencia de un borde o un cambio fuerte en la imagen (por ejemplo, borde de un objeto).
- Un gradiente pequeño indica que la imagen es uniforme en esa región.

3. Pesos adaptativos basados en la imagen $e^{-|\partial_x I_t|}$ y $e^{-|\partial_y I_t|}$

- Actúan como factores de atenuación que controlan cuánto se penaliza la variación en la profundidad según el contenido de la imagen.
- Si $|\partial_x I_t|$ es grande (borde fuerte en la imagen), entonces $e^{-|\partial_x I_t|}$ es pequeño, lo que reduce la penalización en los bordes y permite cambios abruptos en la profundidad.
- Si $|\partial_x I_t|$ es pequeño (región uniforme en la imagen), entonces $e^{-|\partial_x I_t|}$ es grande, lo que aumenta la penalización y fuerza una profundidad más suave.

El término $|\partial_x d_t^*| \cdot e^{-|\partial_x I_t|}$ impone suavidad en la profundidad en regiones de baja textura, pero permite cambios en profundidad en regiones con bordes fuertes.

+ 3 pts Click here to replace this description.

+ 2.5 pts Click here to replace this description.

+ 2 pts Click here to replace this description.

+ 1.5 pts Click here to replace this description.

✓ + 1 pt Click here to replace this description.

+ 0.5 pts Click here to replace this description.

+ 5 pts Correcto!

+ 0 pts 1. **Error absoluto de la profundidad** $|d_i - \hat{d}_i|$

- Este término mide la diferencia entre la profundidad real y la predicha en cada píxel.
- Un valor grande indica una mala predicción, mientras que un valor pequeño indica una buena precisión en la estimación de profundidad.

2. **Margen dinámico dependiente de la imagen** $\tau_i = \gamma e^{-\beta \nabla I_i}$

- Este margen define una tolerancia al error, adaptándose según la estructura de la imagen.
- ∇I_i (gradiente local de la imagen), mide cuánto cambia la intensidad de los píxeles en la vecindad del píxel i .
- Si ∇I_i es alto (bordes), $e^{-\beta \nabla I_i}$ es pequeño, reduciendo el margen τ_i y haciendo que el modelo penalice más los errores.
- Si ∇I_i es bajo (regiones uniformes), $e^{-\beta \nabla I_i}$ es grande, aumentando el margen τ_i y permitiendo más tolerancia al error.
- γ (factor de escala): Controla la magnitud máxima del margen dinámico.
- β (control de sensibilidad): Ajusta qué tan rápido disminuye el margen cuando hay cambios en la imagen.

3. **Penalización del error sin el margen** $\max(0, |d_i - \hat{d}_i| - \tau_i)$

- Si el error es menor que τ_i : No hay penalización $\max(0, x) = 0$, permitiendo cierta flexibilidad en la predicción.
- Si el error supera τ_i : Se aplica penalización proporcional al exceso del margen.
- Esto permite tolerar pequeños errores en regiones uniformes, donde la profundidad es menos ambigua, pero impone una penalización más estricta en bordes y estructuras complejas, donde la precisión es más crucial.

4. **Promedio sobre todos los píxeles** $\frac{1}{N} \sum_{i=1}^N (\cdot)$

- La pérdida final es el promedio del término anterior sobre todos los píxeles en la imagen.

+ 4.5 pts Click here to replace this description.

+ 4 pts Click here to replace this description.

+ 3 pts Click here to replace this description.

✓ + 2.5 pts Click here to replace this description.

+ 2 pts Click here to replace this description.

+ 1.5 pts Click here to replace this description.

+ 1 pt Click here to replace this description.

+ 0.5 pts Click here to replace this description.

+ 0.25 pts Click here to replace this description.

Question 7

✓ + 5 pts Correcto!

+ 0 pts

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Conv3x3(nn.Module):
    def __init__(self, in_chan, out_chan):
        super(Conv3x3, self).__init__()
        self.conv = nn.Conv2d(in_chan, out_chan,
                               kernel_size=3,
                               stride=1,
                               padding=1,
                               bias=False)
        self.bn = nn.BatchNorm2d(out_chan)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        x = self.conv(x)
        x = self.bn(x)
        x = self.relu(x)
        return x

class ASPP(nn.Module):
    def __init__(self, in_chan, out_chan, dilations=(12, 24, 36)):
        super(ASPP, self).__init__()

        self.branch1 = nn.Sequential(
            nn.Conv2d(in_chan, out_chan, kernel_size=1, bias=False),
            nn.BatchNorm2d(out_chan),
            nn.ReLU(inplace=True)
        )

        self.branch2 = nn.Sequential(
            nn.Conv2d(in_chan, out_chan, kernel_size=3,
                      stride=1, padding=dilations[0], dilation=dilations[0],
                      bias=False),
            nn.BatchNorm2d(out_chan),
            nn.ReLU(inplace=True)
        )

        self.branch3 = nn.Sequential(
            nn.Conv2d(in_chan, out_chan, kernel_size=3,
                      stride=1, padding=dilations[1], dilation=dilations[1],
                      bias=False),
            nn.BatchNorm2d(out_chan),
            nn.ReLU(inplace=True)
        )

        self.branch4 = nn.Sequential(
            nn.Conv2d(in_chan, out_chan, kernel_size=3,
                      stride=1, padding=dilations[2], dilation=dilations[2],
                      bias=False),
```

```

        nn.BatchNorm2d(out_chan),
        nn.ReLU(inplace=True)
    )

    self.global_pool = nn.Sequential(
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Conv2d(in_chan, out_chan, kernel_size=1, bias=False),
        nn.BatchNorm2d(out_chan),
        nn.ReLU(inplace=True)
    )

    self.conv1 = nn.Conv2d(out_chan * 5, out_chan, kernel_size=1, bias=False)
    self.bn1 = nn.BatchNorm2d(out_chan)
    self.relu = nn.ReLU(inplace=True)

def forward(self, x):
    b, c, h, w = x.shape

    feat1 = self.branch1(x)
    feat2 = self.branch2(x)
    feat3 = self.branch3(x)
    feat4 = self.branch4(x)

    gp = self.global_pool(x)
    gp = F.interpolate(gp, size=(h, w), mode='bilinear', align_corners=False)

    x = torch.cat([feat1, feat2, feat3, feat4, gp], dim=1)
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)

    return x

class DeepLabV3Plus(nn.Module):
    def __init__(self,
                 backbone,
                 n_low_level_chan,
                 n_high_level_chan,
                 n_classes):
        super(DeepLabV3Plus, self).__init__()

        self.backbone = backbone
        self.aspp = ASPP(n_high_level_chan, 256, dilations=(12, 24, 36))

        self.conv_low_level = nn.Sequential(
            nn.Conv2d(n_low_level_chan, 48, kernel_size=1, bias=False),
            nn.BatchNorm2d(48),
            nn.ReLU(inplace=True)
        )

        self.decoder_block = nn.Sequential(
            Conv3x3(48 + 256, 256),
            Conv3x3( 256, 256)
        )

        self.classifier = nn.Conv2d(256, n_classes, kernel_size=1, bias=False)

```

```
def forward(self, x):
    in_size = x.shape[2:]

    low_level_feat, high_level_feat = self.backbone(x)

    x = self.aspp(high_level_feat)
    x = F.interpolate(x, size=low_level_feat.shape[2:], mode='bilinear', align_corners=False)

    low_level_feat = self.conv_low_level(low_level_feat)
    x = torch.cat([x, low_level_feat], dim=1)

    x = self.decoder_block(x)
    x = F.interpolate(x, size=in_size, mode='bilinear', align_corners=False)

    x = self.classifier(x)
    return x
```

+ 4 pts Click here to replace this description.

+ 3 pts Click here to replace this description.

+ 0.5 pts Click here to replace this description.

+ 2 pts Click here to replace this description.

+ 0.5 pts Click here to replace this description.

Question 8

Puntitos

2 / 4 pts

+ 4 pts 🐱

+ 3 pts 😊

✓ + 2 pts 😊

+ 1 pt 😊

+ 0 pts tu no

Apellidos: Méndez Lázaro

Nombres: Luis Fernando

Fecha: 11/02/2025

Nota:

Indicaciones:

La Duración es de 40 minutos.

La evaluación consta de 10 preguntas.

Las preguntas de opción múltiple valen 1.5 punto.

1. ¿Cuál de la ventaja principal de DeepLabV3+ sobre PSPNet en la fusión de características?

- a) DeepLabV3+ emplea un módulo de decoder que únicamente integra high-level features para reducir la redundancia, lo que minimiza el costo computacional, pero a costa de una menor precisión en los bordes.
- ☒ b) DeepLabV3+ combina el uso de ASPP para extraer características multiescala con un módulo de decoder que integra explícitamente low-level features para refinar las segmentaciones, superando las limitaciones de PSPNet en la precisión de los bordes.
- c) DeepLabV3+ reemplaza el uso de atrous spatial pyramid pooling (ASPP) por una serie de capas fully-connected, permitiendo capturar dependencias globales sin fusionar explícitamente información local.
- d) DeepLabV3+ se basa en el uso exclusivo de atrous convolutions sin incorporar mecanismos de fusión, lo que facilita el procesamiento de imágenes de alta resolución, pero limita la integración de escalas.
- e) DeepLabV3+ introduce un módulo de atención que descarta low-level features, enfocándose únicamente en la información semántica profunda para mejorar la segmentación en regiones homogéneas.

2. ¿Cuál es la diferencia de utilizar InfoNCE en comparación con Triplet Loss?

- a) InfoNCE se diferencia de Triplet Loss al emplear una normalización softmax sobre un amplio conjunto de muestras negativas, lo que permite calcular la información mutua exacta entre las representaciones, en contraste con el uso de un margen fijo en Triplet Loss.
- b) InfoNCE y Triplet Loss son esencialmente equivalentes, ya que ambos optimizan la separación entre muestras positivas y negativas sin incorporar mecanismos de normalización, haciendo que sus rendimientos sean comparables.

☒ c) A diferencia de Triplet Loss, InfoNCE descarta el uso de muestras negativas, concentrándose únicamente en maximizar la similitud entre el anchor y el positivo, lo que simplifica el entrenamiento, pero reduce la diversidad en las representaciones.

d) InfoNCE incorpora una penalización adicional para evitar el sobreajuste en las muestras negativas, reduciendo la sensibilidad al tamaño del batch en comparación con Triplet Loss.

e) InfoNCE maximiza la probabilidad de identificar correctamente el par positivo entre múltiples distractores, operando en un espacio probabilístico que refleja la información mutua, a diferencia de Triplet Loss que se centra en empujar a las muestras negativas a distancias mayores sin considerar la distribución global.

3. Dada la función Softmax definida como

$$\text{Softmax}_i(z) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

donde τ es el parámetro temperature, ¿cuál es el efecto de aumentar el valor de τ en la distribución de probabilidades resultante?

- a) Aumentar τ intensifica las diferencias entre las probabilidades, elevando la probabilidad de la clase con mayor logit y haciendo la distribución más puntiaguda.
- b) Disminuir τ conduce a una distribución más uniforme, ya que los logits se normalizan de manera equitativa entre todas las clases.
- ☒ c) Aumentar τ suaviza la distribución, reduciendo las diferencias entre las probabilidades asignadas a cada clase y haciendo que la distribución sea más uniforme.
- d) Reducir τ suaviza la distribución, lo que genera mayor incertidumbre al asignar probabilidades similares a todas las clases.
- e) Modificar τ solo escala los logits sin alterar la forma relativa de la distribución, por lo que su efecto es meramente numérico.

$$\mathcal{L}_q = - \log \frac{\exp(q \cdot k / s)}{\sum}$$

4. Analice las implicaciones del diseño de la función loss en contrastive learning, discutiendo cómo elementos como el parámetro temperature y la definición de la distancia en el feature space afectan la capacidad del modelo para capturar similitudes y diferencias relevantes. [5 pts]

El parámetro temperature en modelos como MoCo suaviza o agudiza la distribución de probabilidades afectando directamente su rendimiento. Por otro lado, la distancia en modelos como SimCLR

$$\mathcal{L}_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j))}{\sum_{k=1}^{2N} \mathbb{1}[k \neq i] \exp(\text{sim}(z_i, z_k))} \quad \text{donde} \quad \text{sim}(z_i, z_j) = \frac{z_i \cdot z_j}{\|z_i\| \|z_j\|}$$

relevantes para maximizar la cercanía entre claves positivas y repeliendo claves negativas, afectando su aprendizaje de buenas representaciones (embedding) y por lo tanto su rendimiento.

5. Se define Edge-aware smooth loss como:

$$L_s = |\partial_x d_t^*| \cdot e^{-|\partial_x l_t|} + |\partial_y d_t^*| \cdot e^{-|\partial_y l_t|}$$

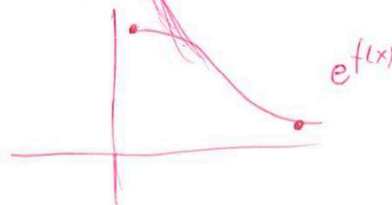
donde:

- d_t^* es la profundidad predicha en la muestra t
- l_t es la imagen correspondiente en la muestra t
- ∂_x y ∂_y denotan las derivadas direccionales en los ejes horizontal y vertical, respectivamente.

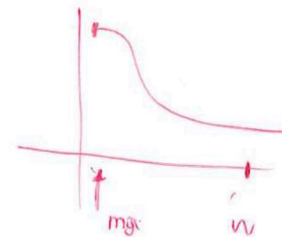
Interprete correctamente cada componente de la función. Justifique.

[5 pts]

L_s intenta suavizar el mapa de profundidad para alto contraste. Según la ecuación tenemos, este comportamiento ^{esto} _{caracterizado}



Se le asigna una mayor pérdida cuando el contraste es elevado



6. Se define Dynamic Margin Depth Loss (DMDL) para estimación de profundidad:

$$\mathcal{L}_{DD}(d, \hat{d}) = \frac{1}{N} \sum_{i=1}^N \max(0, |d_i - \hat{d}_i| - \tau_i)$$

con

$$\tau_i = \gamma \cdot \exp(-\beta \|\nabla I_i\|)$$

donde:

- d_i y $\hat{d}_i$ son, respectivamente, la profundidad real y la predicha en el píxel i ,
- I es la imagen de entrada,
- ∇I_i representa el gradiente (local) de I en el píxel i ,
- γ y β son constantes.

Interprete correctamente cada componente de la función. Justifique.

[5 pts]

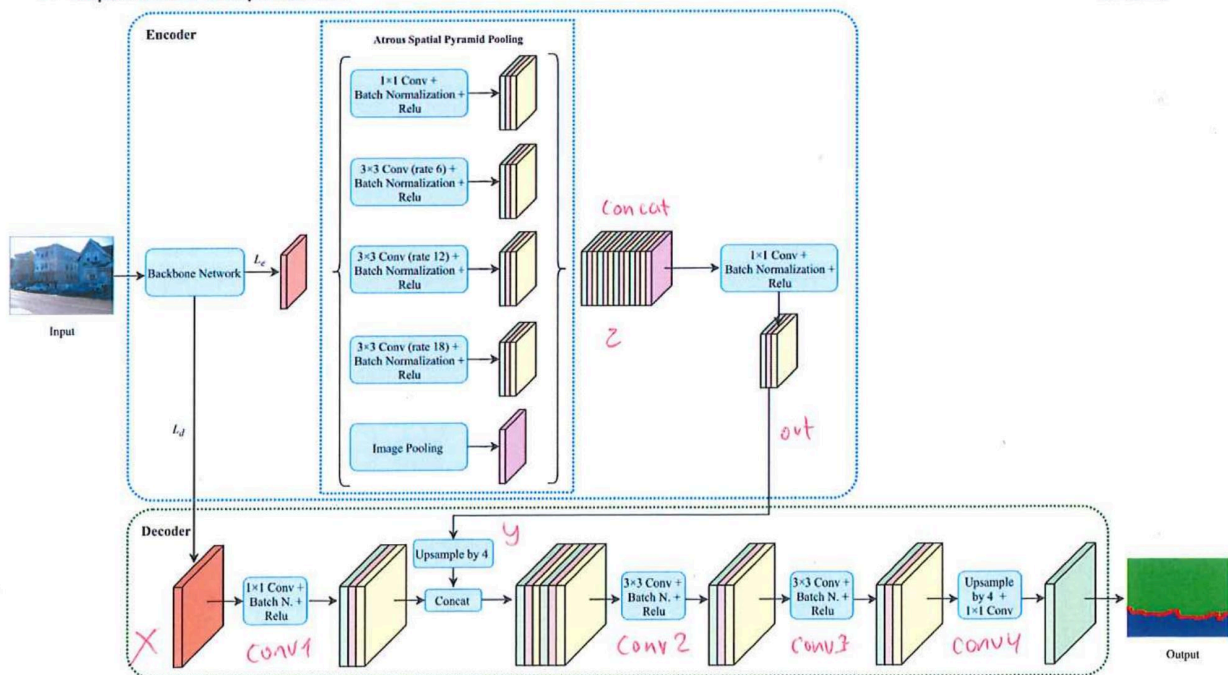
En general, la función DMDL cuantifica una ponderación de errores ~~entre~~ de "similitud" entre los píxeles de la imagen I

$$\frac{1}{N} \sum_{i=1}^N \max(0, |d_i - \hat{d}_i| - \tau_i)$$

Esta comparación se apoya de información local mediante τ_i por medio de ∇I_i exceptuando de valores negativos mediante $\max(0, |d_i - \hat{d}_i| - \tau_i)$

7. Implemente DeepLabV3+.

[5 pts]



Asuma que puede importar el backbone de su elección.

Encoder :

init:

```

backbone = resnet50() # quitar últimas capas, channels = Resnet50 out_channels
conv1 = Conv2d(channels, channel, 1)
conv2 = Conv2d(channels, channel, 3, dilated=6)
conv3 = Conv2d(channels, channel, 3, dilated=12)
conv4 = Conv2d(channels, channel, 3, dilated=18)
pool = AvgPool2d(channels, channel)
  
```

forward(x)

```

Le = backbone.layer2(x), Ld = backbone.layer2(x) # Feature map intermedio
out1 = F.relu(nn.BatchNorm2d(conv1(Le)))
out2 = F.relu(nn.BatchNorm2d(conv2(Le)))
out3 = F.relu(nn.BatchNorm2d(conv3(Le)))
out4 = F.relu(nn.BatchNorm2d(conv4(Le)))
out5 = nn.BatchNorm2d(conv4(Le)) pool(Le)
# up sample outs :)
z = cat([out1, out2, out3, out4, out5])
out = F.relu(nn.BatchNorm2d(conv1(z)))
return out, Ld
  
```

Decoder :

init:

```

conv1 = Conv2d(...) # 1x1
conv2 = Conv2d(...) # 3x3
conv3 = Conv2d(...) # 3x4
upsample4 conv4 = TransposeConv2d(1) # upsample
  
```

forward(x,y) # out, Ld

```

x = conv1(x) F.relu(BatchNorm2d(conv1(x)))
x = cat([x, upsample4(y)])
x = conv2(x) F.relu(BatchNorm2d(conv2(x)))
x = F.relu(BatchNorm2d(conv3(x)))
x = conv1 Conv1(upsample4(x))
return x
  
```

DeepLab implementation

↓
a través

4 de 4